

✓ Carga de librerías

```

1 !pip install requests      #Hacer llamadas http a paginas de la red
2 !pip install tsplib95      #Modulo para las instancias del problema del TSP

```

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2024.7.4)
Collecting tsplib95
 Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.12/dist-packages (from tsplib95) (8.3.1)
Collecting Deprecated~1.2.9 (from tsplib95)
 Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
Collecting networkx~2.1 (from tsplib95)
 Downloading networkx-2.8.8-py3-none-any.whl.metadata (5.1 kB)
Collecting tabulate~0.8.7 (from tsplib95)
 Downloading tabulate-0.8.10-py3-none-any.whl.metadata (25 kB)
Collecting wrapt<2,>=1.10 (from Deprecated~1.2.9->tsplib95)
 Downloading wrapt-1.17.3-cp312-cp312-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl.metadata (88 kB)
 Downloading tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
 Downloading Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
 Downloading networkx-2.8.8-py3-none-any.whl (2.0 MB)
 2.0/2.0 MB 63.6 MB/s eta 0:00:00
 Downloading tabulate-0.8.10-py3-none-any.whl (29 kB)
 Downloading wrapt-1.17.3-cp312-cp312-manylinux1_x86_64.manylinux2_28_x86_64.manylinux2_5_x86_64.whl (88 kB)
 88.0/88.0 kB 5.8 MB/s eta 0:00:00
Installing collected packages: wrapt, tabulate, networkx, Deprecated, tsplib95
 Attempting uninstall: wrapt
 Found existing installation: wrapt 2.1.1
 Uninstalling wrapt-2.1.1:
 Successfully uninstalled wrapt-2.1.1
 Attempting uninstall: tabulate
 Found existing installation: tabulate 0.9.0
 Uninstalling tabulate-0.9.0:
 Successfully uninstalled tabulate-0.9.0
 Attempting uninstall: networkx
 Found existing installation: networkx 3.6.1
 Uninstalling networkx-3.6.1:
 Successfully uninstalled networkx-3.6.1
 ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This is b
 bigframes 2.36.0 requires tabulate>=0.9, but you have tabulate 0.8.10 which is incompatible.
 scikit-image 0.25.2 requires networkx>=3.0, but you have networkx 2.8.8 which is incompatible.
 mapclassify 2.10.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
 momepy 0.11.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
 spopt 0.7.0 requires networkx>=3.2, but you have networkx 2.8.8 which is incompatible.
 Successfully installed Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8.10 tsplib95-0.7.1 wrapt-1.17.3

✓ Carga de los datos del problema

```

1 import urllib.request #Hacer llamadas http a paginas de la red
2 import tsplib95        #Modulo para las instancias del problema del TSP
3 import math            #Modulo de funciones matematicas. Se usa para exp
4 import random          #Para generar valores aleatorios
5 import gzip
6
7 #http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
8 #Documentacion :
9 # http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf
10 # https://tsplib95.readthedocs.io/en/stable/pages/usage.html
11 # https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
12 # https://pypi.org/project/tsplib95/
13
14 #Descargamos el fichero de datos(Matriz de distancias)
15 # file = "swiss42.tsp" ;
16 # urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/swiss42.tsp.gz", fil
17 # !gzip -d swiss42.tsp.gz #Descomprimir el fichero de datos
18
19 #Coordendas 51-city problem (Christofides/Eilon)
20 #file = "eil51.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/
21
22 #Coordenadas - 48 capitals of the US (Padberg/Rinaldi)
23 #file = "att48.tsp" ; urllib.request.urlretrieve("http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/
24
25 # Estaba caido el server, por lo que lo hacemos usando esta URL:
26 file = "swiss42.tsp" ;
27 urllib.request.urlretrieve("https://raw.githubusercontent.com/coin-or/jorlib/refs/heads/master/jorlib-core/s

```

```
28
29
```

```
('swiss42.tsp', <http.client.HTTPMessage at 0x784d56e54200>)
```

```
1 #Carga de datos y generación de objeto problem
2 #####
3 problem = tsplib95.load(file)
4
5 #Nodos
6 Nodos = list(problem.get_nodes())
7
8 #Aristas
9 Aristas = list(problem.get_edges())
10
11
```

```
1 Aristas
```

```
[(0, 0),
(0, 1),
(0, 2),
(0, 3),
(0, 4),
(0, 5),
(0, 6),
(0, 7),
(0, 8),
(0, 9),
(0, 10),
(0, 11),
(0, 12),
(0, 13),
(0, 14),
(0, 15),
(0, 16),
(0, 17),
(0, 18),
(0, 19),
(0, 20),
(0, 21),
(0, 22),
(0, 23),
(0, 24),
(0, 25),
(0, 26),
(0, 27),
(0, 28),
(0, 29),
(0, 30),
(0, 31),
(0, 32),
(0, 33),
(0, 34),
(0, 35),
(0, 36),
(0, 37),
(0, 38),
(0, 39),
(0, 40),
(0, 41),
(1, 0),
(1, 1),
(1, 2),
(1, 3),
(1, 4),
(1, 5),
(1, 6),
(1, 7),
(1, 8),
(1, 9),
(1, 10),
(1, 11),
(1, 12),
(1, 13),
(1, 14),
(1, 15),
```

```

NOMBRE: swiss42
TIPO: TSP
COMENTARIO: 42 Staedte Schweiz (Fricker)
DIMENSION: 42
EDGE_WEIGHT_TYPE: EXPLICIT
EDGE_WEIGHT_FORMAT: FULL_MATRIX
EDGE_WEIGHT_SECTION
0 15 30 23 32 55 33 37 92 114 92 110 96 90 74 76 82 72 78 82 159 122 131 206 112 57 28 43 70 4
15 0 34 23 27 40 19 32 93 117 88 100 87 75 63 67 71 69 62 63 96 164 132 131 212 106 44 33 5
30 34 0 11 18 57 36 65 62 84 64 89 76 93 95 100 104 98 57 88 99 130 100 101 179 86 51 4 18 4
23 23 11 0 11 48 26 54 70 94 69 75 75 84 84 89 92 89 54 78 99 141 111 109 89 89 11 11 11 54
32 27 18 11 0 40 20 58 67 92 61 78 65 76 83 89 91 95 43 72 110 141 116 105 190 81 34 19 35 1
55 40 57 48 40 0 23 55 96 123 78 75 36 36 66 66 63 95 34 34 137 174 156 129 224 90 15 59 75
33 19 36 26 20 23 0 45 85 111 75 82 69 60 63 70 71 85 44 52 115 161 136 122 210 91 25 37 54
37 32 65 54 58 55 45 0 124 149 118 126 113 80 42 42 40 40 87 87 94 158 158 163 242 135 65 6
92 93 62 70 67 96 85 124 0 28 29 68 63 122 148 155 156 159 67 129 148 78 80 39 129 46 82 65
114 117 84 94 92 123 111 149 28 0 54 91 88 150 174 181 182 181 95 157 159 50 65 27 102 65 11
92 88 64 69 61 78 75 118 29 54 0 39 34 99 134 142 141 157 44 110 161 103 109 52 154 22 63 6
110 100 89 89 78 75 82 126 68 91 39 0 14 80 129 139 135 167 39 98 187 136 148 81 186 28 61 9
96 87 76 75 65 62 69 113 63 88 34 14 0 72 117 128 124 153 26 88 174 136 142 82 187 32 48 79
90 75 93 84 76 36 60 80 122 150 99 80 72 0 59 71 63 116 56 25 170 201 189 151 252 104 44 95
74 63 95 84 83 56 63 42 148 174 134 129 117 59 0 11 8 63 93 35 135 223 195 184 273 146 71 9

```

```

1 #Probamos algunas funciones del objeto problem
2
3 #Distancia entre nodos
4 problem.get_weight(0, 1)
5
6 #Todas las funciones
7 #Documentación: https://tsplib95.readthedocs.io/en/v0.6.1/modules.html
8
9 #dir(problem)

```

15

▼ Funcionas basicas

```

1 #Funcionas basicas
2 #####
3
4 #Se genera una solucion aleatoria con comienzo en en el nodo 0
5 def crear_solucion(Nodos: list) -> list:
6     solucion = [Nodos[0]]
7     # Se hace este truco para evitar bucles.
8     for n in Nodos[1:]:
9         # Esto nos vale para quitar la primera ciudad de la primera iteración "set(Nodos) - set({Nodos[0]})"
10        # Esto nos permite ir quitando las que ya teníamos en la solución "- set(solucion)"
11        solucion = solucion + [random.choice(list(set(Nodos) - set({Nodos[0]}) - set(solucion)))]
12    return solucion
13
14 #Devuelve la distancia entre dos nodos
15 def distancia(a: int, b: int, problem: tsplib95.models.StandardProblem) -> int:
16    return problem.get_weight(a,b)
17
18 #Devuelve la distancia total de una trayectoria/solucion
19 def distancia_total(solucion: list, problem: tsplib95.models.StandardProblem) -> int:
20    distancia_total = 0
21    for i in range(len(solucion) - 1):
22        distancia_total += distancia(solucion[i], solucion[i+1], problem)
23    return distancia_total + distancia(solucion[len(solucion) - 1], solucion[0], problem)
24
25 sol_temporal = crear_solucion(Nodos)
26
27 distancia_total(sol_temporal, problem), sol_temporal

```

```

(5150,
[0,
28,
32,
40,
5,
14,
36,
15,
6,
11,
35,
8,
13,
24,
39,
1,
33,
9,

```

```

20,
26,
19,
22,
29,
3,
31,
18,
2,
16,
4,
30,
21,
37,
23,
17,
10,
41,
7,
12,
38,
25,
27,
34])

```

✓ BUSQUEDA ALEATORIA

```

1 #####
2 # BUSQUEDA ALEATORIA
3 #####
4
5 def busqueda_aleatoria(problem, N):
6     #N es el numero de iteraciones
7     Nodos = list(problem.get_nodes())
8
9     mejor_solucion = []
10    #mejor_distancia = 10e100                #Inicializamos con un valor alto
11    mejor_distancia = float('inf')          #Inicializamos con un valor alto
12
13    for i in range(N):
14        solucion = crear_solucion(Nodos)    #Genera una solucion aleatoria
15        distancia = distancia_total(solucion, problem) #Calcula el valor objetivo(distancia total)
16
17        if distancia < mejor_distancia:
18            mejor_solucion = solucion        #Compara con la mejor obtenida hasta ahora
19            mejor_distancia = distancia
20
21
22    print("Mejor solución:" , mejor_solucion)
23    print("Distancia      :" , mejor_distancia)
24    return mejor_solucion
25
26
27 #Busqueda aleatoria con 5000 iteraciones
28 solucion = busqueda_aleatoria(problem, 10000)

```

```

Mejor solución: [0, 13, 9, 39, 10, 22, 11, 30, 8, 41, 12, 23, 29, 21, 40, 24, 19, 37, 26, 5, 18, 28, 3, 36, 31, 1
Distancia      : 3382

```

✓ BUSQUEDA LOCAL

```

1 #####
2 # BUSQUEDA LOCAL
3 #####
4 def genera_vecina(solucion):
5     #Generador de soluciones vecinas: 2-opt (intercambiar 2 nodos) Si hay N nodos se generan (N-1)x(N-2)/2 sol
6     #Se puede modificar para aplicar otros generadores distintos que 2-opt
7     #print(solucion)
8     mejor_solucion = []
9     mejor_distancia = 10e100
10    for i in range(1, len(solucion)-1):
11        for j in range(i+1, len(solucion)):
12
13        #Se genera una nueva solución intercambiando los dos nodos i,j:
14        # (usamos el operador + que para listas en python las concatena) : ej.: [1,2] + [3] = [1,2,3]
15        vecina = solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]
16
17    #Se evalua la nueva solución ...

```

```

18     distancia_vecina = distancia_total(vecina, problem)
19
20     #... para guardarla si mejora las anteriores
21     if distancia_vecina <= mejor_distancia:
22         mejor_distancia = distancia_vecina
23         mejor_solucion = vecina
24     return mejor_solucion
25
26
27 #solucion = [1, 47, 13, 41, 40, 19, 42, 44, 37, 5, 22, 28, 3, 2, 29, 21, 50, 34, 30, 9, 16, 11, 38, 49, 10,
28 print("Distancia Solucion Inicial:" , distancia_total(solucion, problem))
29
30
31 nueva_solucion = genera_vecina(solucion)
32 print("Distancia Mejor Solucion Local:", distancia_total(nueva_solucion, problem))
33

```

Distancia Solucion Inicial: 3382
Distancia Mejor Solucion Local: 3069

```

1 #Busqueda Local:
2 # - Sobre el operador de vecindad 2-opt(funcion genera_vecina)
3 # - Sin criterio de parada, se para cuando no es posible mejorar.
4 def busqueda_local(problem):
5     mejor_solucion = []
6
7     #Generar una solucion inicial de referencia(aleatoria)
8     solucion_referencia = crear_solucion(Nodos)
9     mejor_distancia = distancia_total(solucion_referencia, problem)
10
11     iteracion=0           #Un contador para saber las iteraciones que hacemos
12     while(1):
13         iteracion +=1      #Incrementamos el contador
14         #print('#',iteracion)
15
16         #Obtenemos la mejor vecina ...
17         vecina = genera_vecina(solucion_referencia)
18
19         #... y la evaluamos para ver si mejoramos respecto a lo encontrado hasta el momento
20         distancia_vecina = distancia_total(vecina, problem)
21
22         #Si no mejoramos hay que terminar. Hemos llegado a un minimo local(según nuestro operador de vecindad 2
23         if distancia_vecina < mejor_distancia:
24             #mejor_solucion = copy.deepcopy(vecina)    #Con copia profunda. Las copias en python son por referencia
25             mejor_solucion = vecina                    #Guarda la mejor solución encontrada
26             mejor_distancia = distancia_vecina
27
28         else:
29             print("En la iteracion ", iteracion, ", la mejor solución encontrada es:" , mejor_solucion)
30             print("Distancia      :", mejor_distancia)
31             return mejor_solucion
32
33         solucion_referencia = vecina
34
35
36 sol = busqueda_local(problem)

```

En la iteracion 38 , la mejor solución encontrada es: [0, 32, 34, 33, 38, 22, 39, 24, 40, 21, 9, 23, 41, 10, 8,
Distancia : 1577

✓ SIMULATED ANNEALING

```

1 #####
2 # SIMULATED ANNEALING
3 #####
4
5 #Generador de 1 solucion vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
6 #Mejorable eligiendo otra forma de elegir una vecina.
7 def genera_vecina_aleatorio(solucion):
8
9     #Se eligen dos nodos aleatoriamente
10    i,j = sorted(random.sample( range(1,len(solucion)) , 2))
11
12    #Devuelve una nueva solución pero intercambiando los dos nodos elegidos al azar
13    return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + solucion[j+1:]
14
15
16 #Funcion de probabilidad para aceptar peores soluciones
17 def probabilidad(T,d):

```

```

18 if random.random() < math.exp( -1*d / T ) :
19     return True
20 else:
21     return False
22
23 #Funcion de descenso de temperatura
24 def bajar_temperatura(T):
25     return T*0.99

```

```

1 def recocido_simulado(problem, TEMPERATURA ):
2     #problem = datos del problema
3     #T = Temperatura
4
5     solucion_referencia = crear_solucion(Nodos)
6     distancia_referencia = distancia_total(solucion_referencia, problem)
7
8     mejor_solucion = []           #x* del seudocodigo
9     mejor_distancia = 10e100      #F* del seudocodigo
10
11
12     N=0
13     while TEMPERATURA > .0001:
14         N+=1
15         #Genera una solución vecina
16         vecina =genera_vecina_aleatorio(solucion_referencia)
17
18         #Calcula su valor(distancia)
19         distancia_vecina = distancia_total(vecina, problem)
20
21         #Si es la mejor solución de todas se guarda(siempre!!!)
22         if distancia_vecina < mejor_distancia:
23             mejor_solucion = vecina
24             mejor_distancia = distancia_vecina
25
26         #Si la nueva vecina es mejor se cambia
27         #Si es peor se cambia según una probabilidad que depende de T y delta(distancia_referencia - distancia_v
28         if distancia_vecina < distancia_referencia or probabilidad(TEMPERATURA, abs(distancia_referencia - dista
29             #solucion_referencia = copy.deepcopy(vecina)
30             solucion_referencia = vecina
31             distancia_referencia = distancia_vecina
32
33         #Bajamos la temperatura
34         TEMPERATURA = bajar_temperatura(TEMPERATURA)
35
36     print("La mejor solución encontrada es " , end="")
37     print(mejor_solucion)
38     print("con una distancia total de " , end="")
39     print(mejor_distancia)
40     return mejor_solucion
41
42 sol = recocido_simulado(problem, 10000000)

```

La mejor solución encontrada es [0, 32, 20, 33, 34, 28, 30, 29, 9, 40, 23, 41, 25, 11, 18, 27, 8, 10, 12, 26, 6, con una distancia total de 1856